

QUIZ

What is k-fold cross validation and why is it useful?

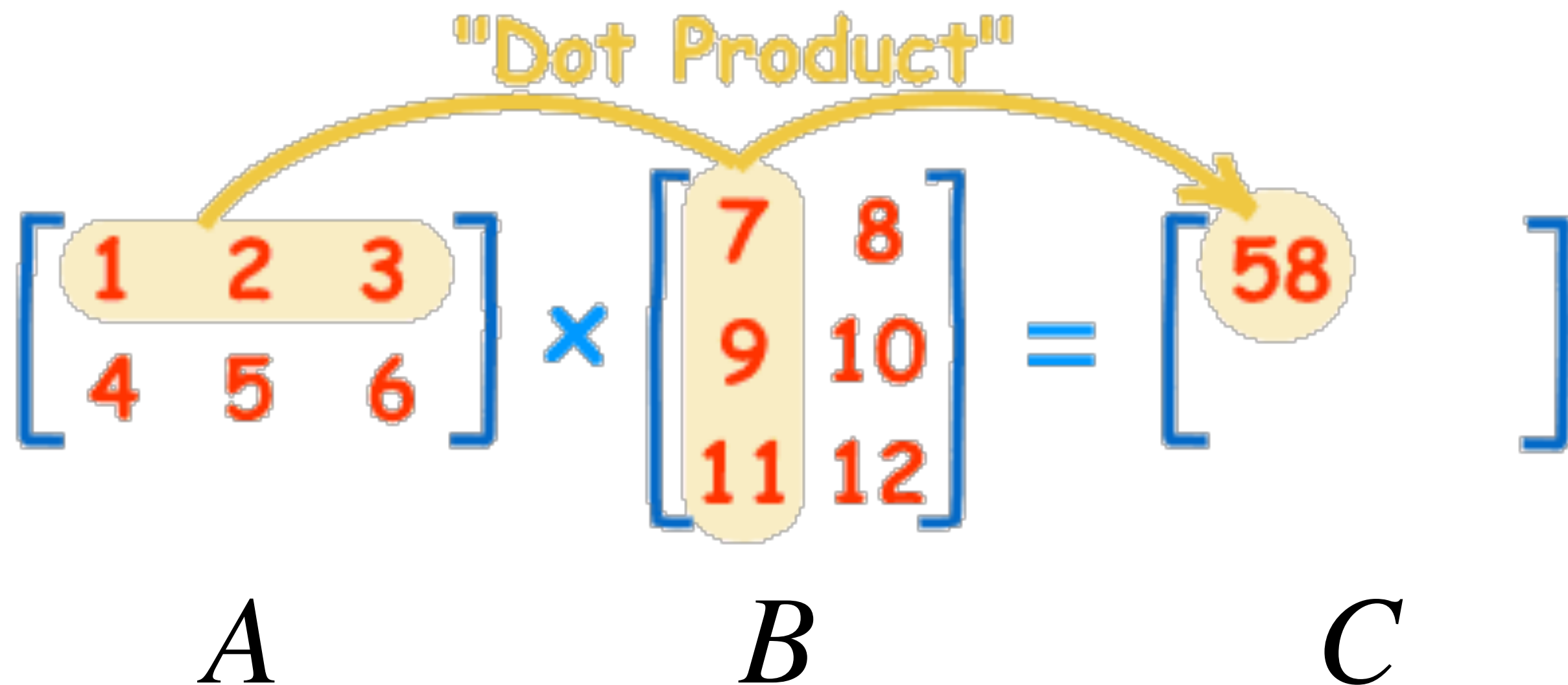
REVIEW OF (THE LAST PART OF) PREVIOUS LECTURE

Matrix transpose: A^T

$$A_{ij}^T = A_{ji}, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

Matrix multiplication: $C = AB$ (A is $n \times k$, B is $k \times m$)

$$C_{ij} = A_i \cdot B_j, \quad \forall 1 \leq i \leq n, 1 \leq j \leq m$$

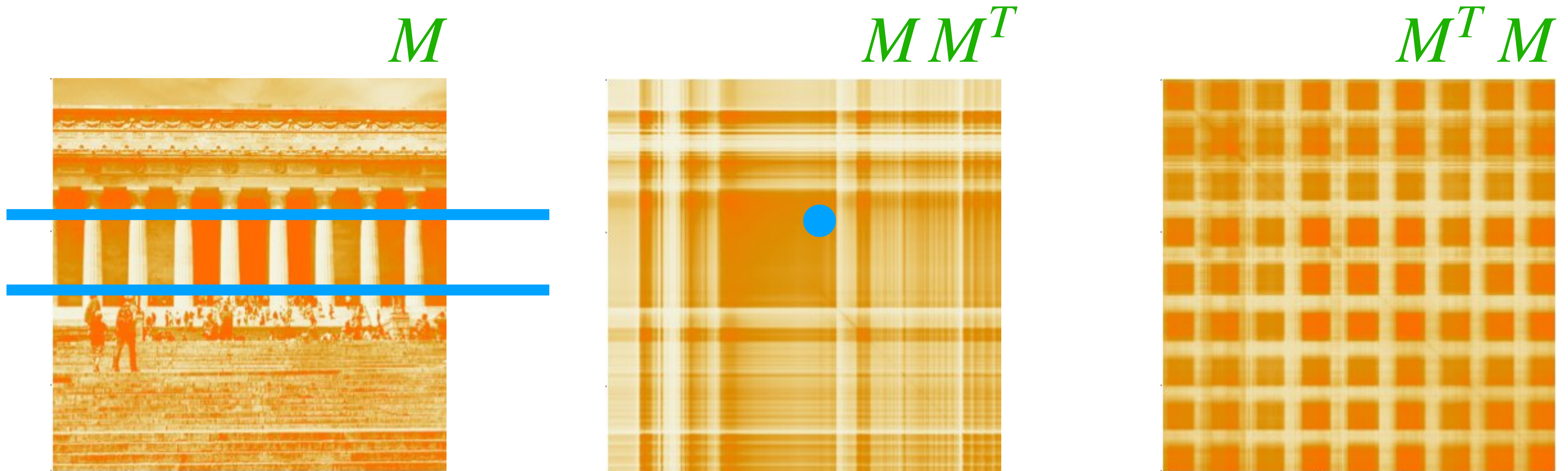


Note that, Dot products measure how “in sync” the two vectors are (like covariance or correlation).

REVIEW OF (THE LAST PART OF) PREVIOUS LECTURE

$A A^T$ measures “in-syncness” among rows (items/points).

$A^T A$ measures “in-syncness” among columns (features/dimensions).



ALGORITHMS FOR MODELING

ALGORITHMS FOR MODELING

This lecture:

- Simple Linear regression
- Multiple linear regression
- Goodness of fit

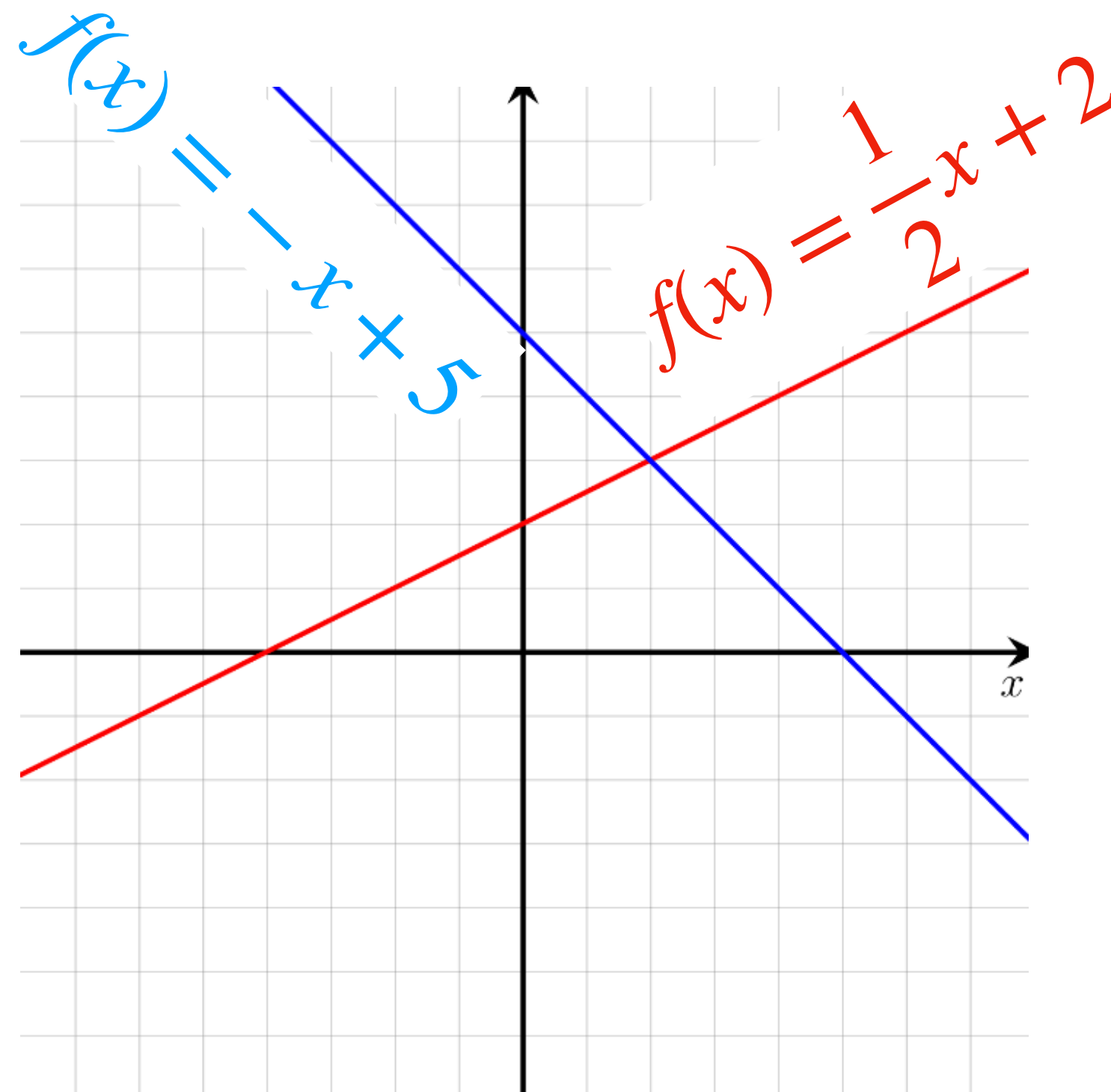
Readings:

- Ch3 from ISLP, James et al. textbook.
- 9.1-9.5 from Skiena's textbook.

LINEAR REGRESSION

Line equation: $f(x) = b + ax$

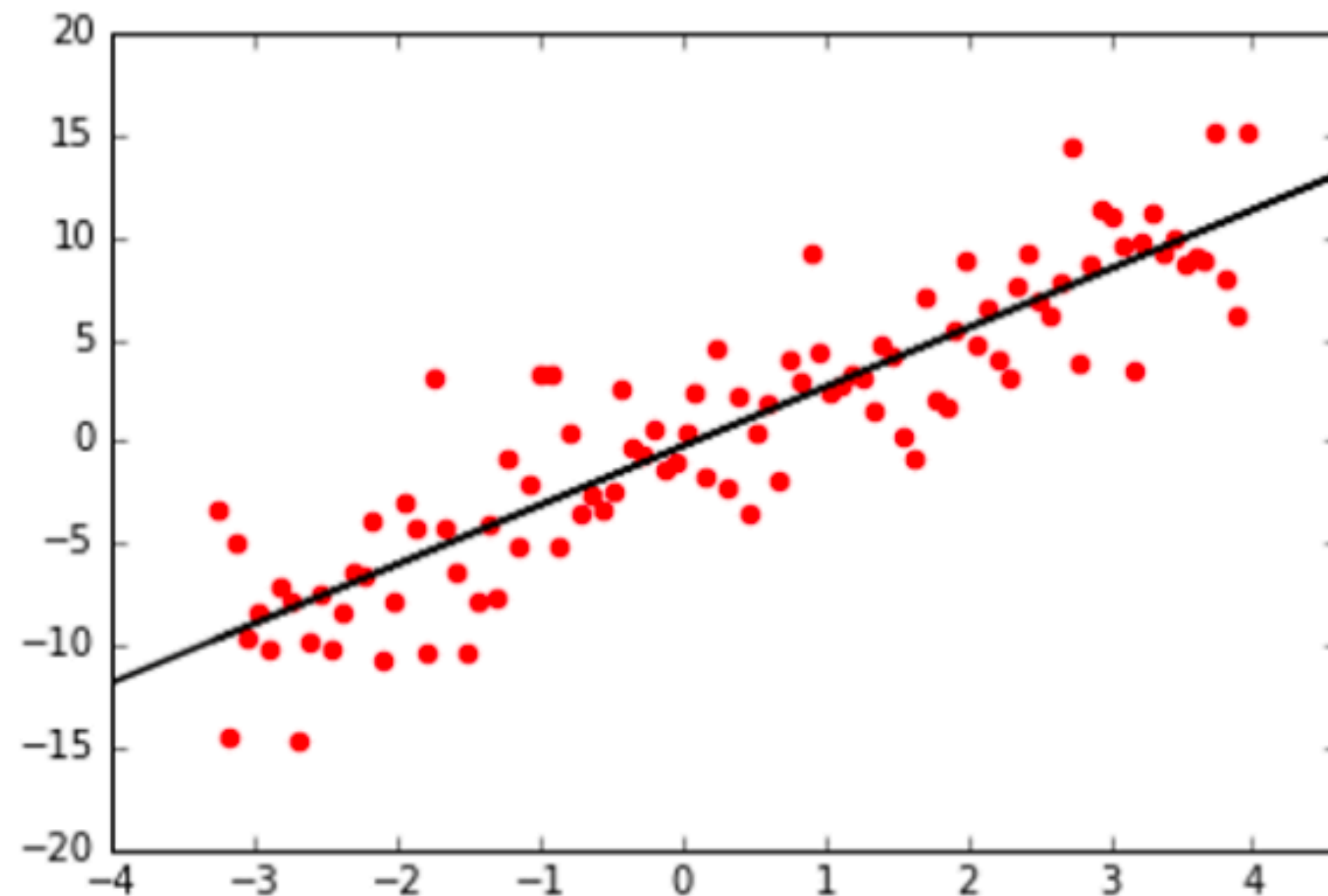
Intercept
Slope



LINEAR REGRESSION

Linear regression in 2D:

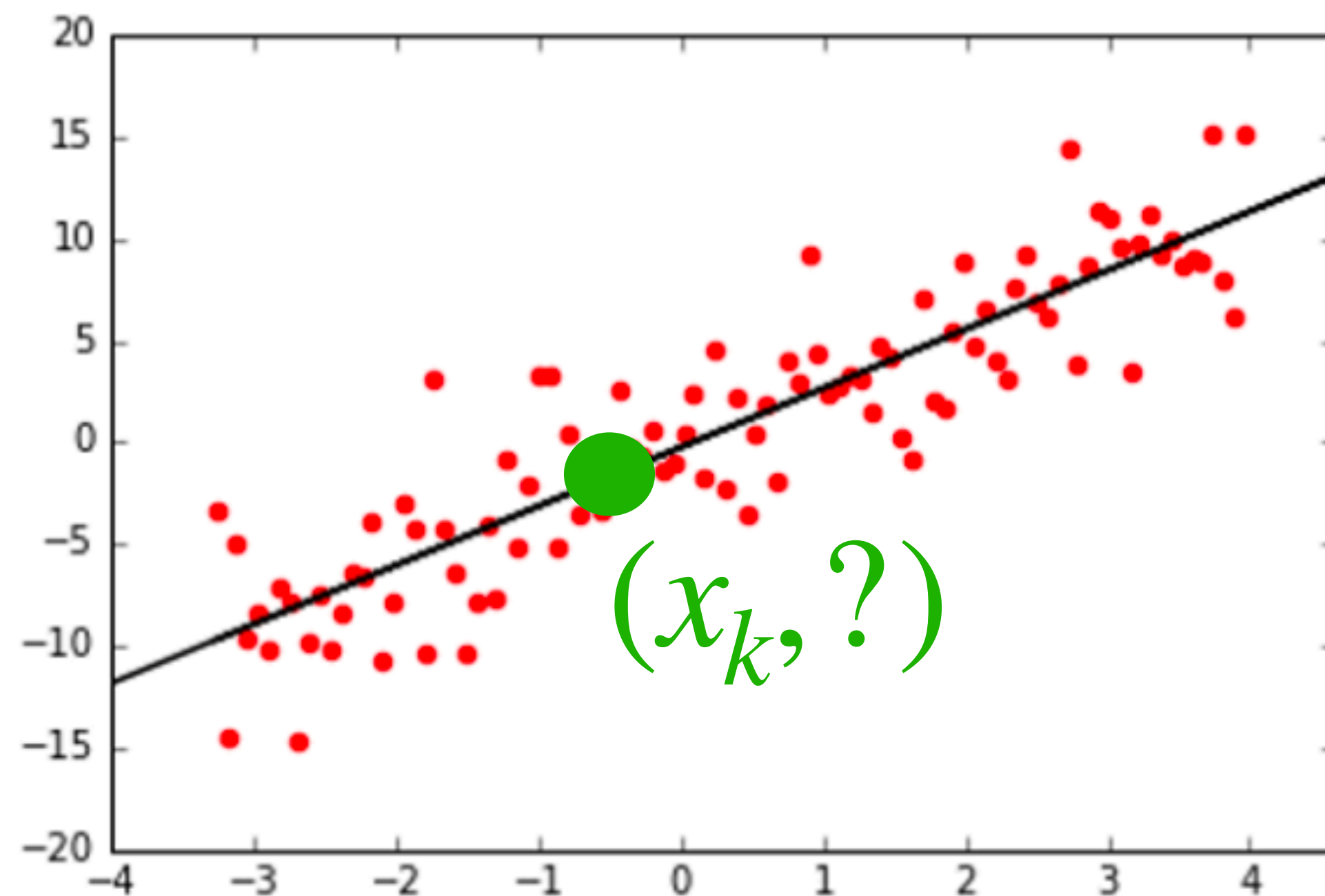
Find $\hat{y} = \hat{w}_0 + \hat{w}_1 x$ that **best** fits the set of points.



LINEAR REGRESSION

Linear regression in 2D:

Find $\hat{y} = \hat{w}_0 + \hat{w}_1 x$ that **best** fits the set of points.



Given x_k of a new point p_k , we predict its y-coord by $\hat{y}_k = \hat{w}_0 + \hat{w}_1 x_k$.

LINEAR REGRESSION

Linear regression in 2D:

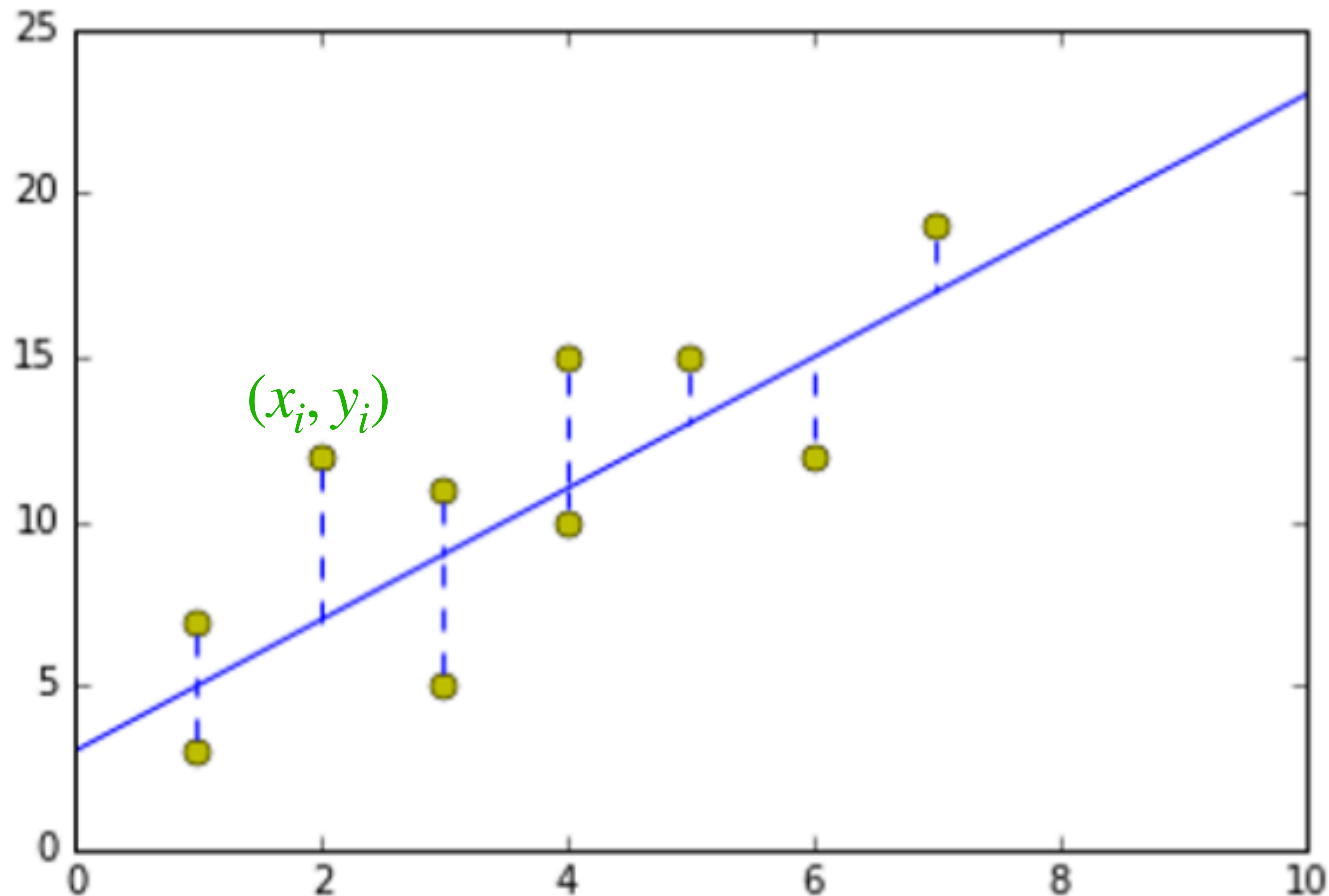
Find $\hat{y} = \hat{w}_0 + \hat{w}_1 x$ that **best** fits the set of points.

How to define **best**?

LINEAR REGRESSION

Residual of data point (x_i, y_i) : $y_i - \hat{y}_i$

$$(\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i)$$

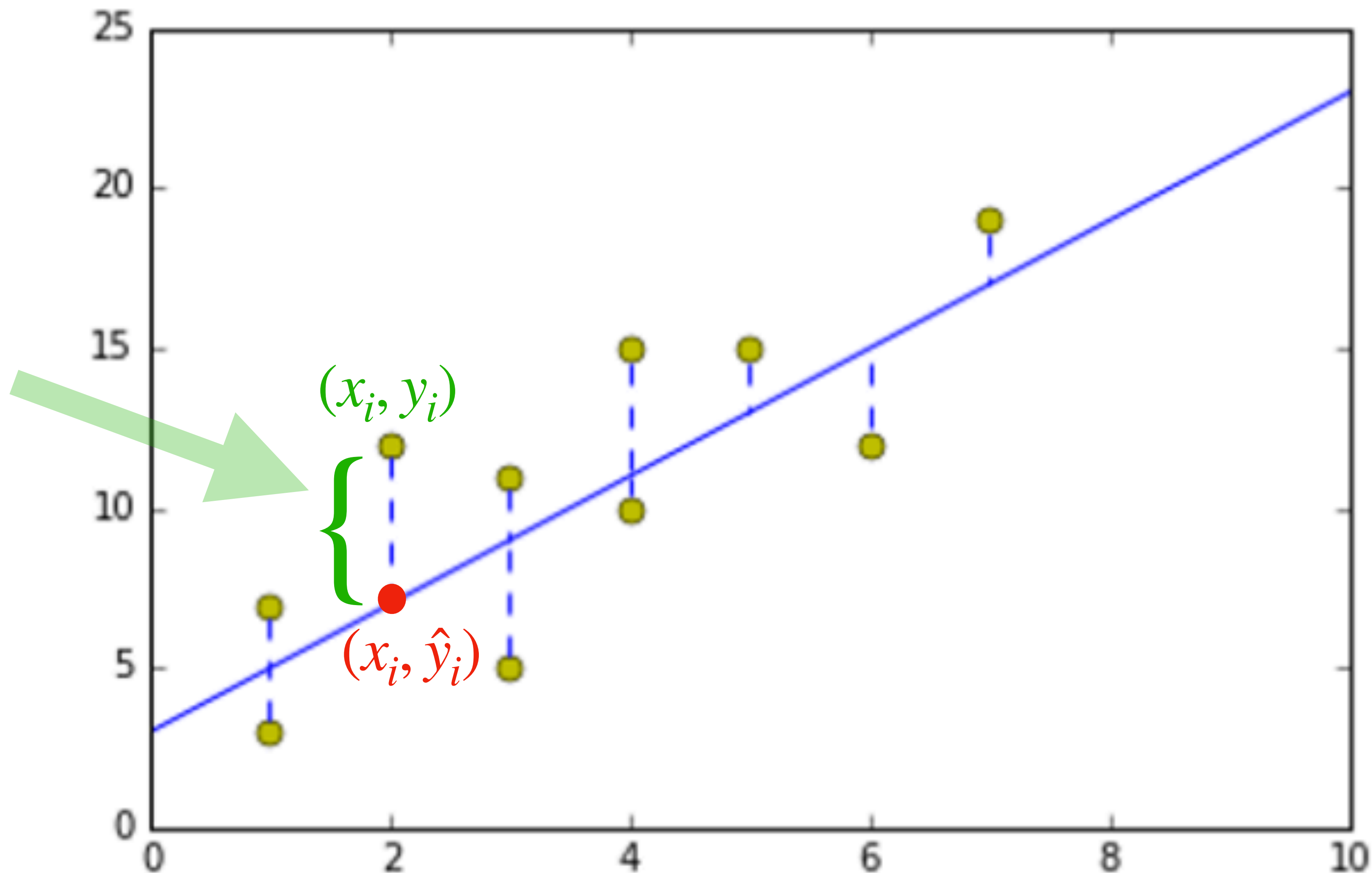


LINEAR REGRESSION

Residual of data point (x_i, y_i) : $y_i - \hat{y}_i$

$$(\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i)$$

Residual



LINEAR REGRESSION

Residual of data point (x_i, y_i) : $y_i - \hat{y}_i$

$$(\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i)$$

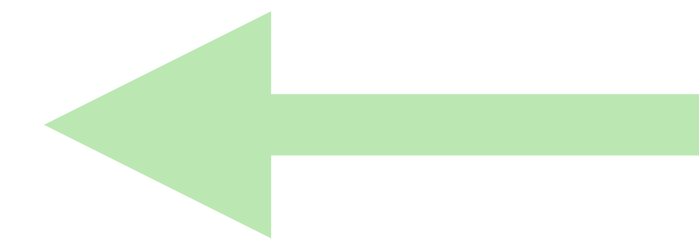
Residual sum of squares:
(RSS) $\sum_{i=1}^n (y_i - \hat{y}_i)^2$

LINEAR REGRESSION

Residual of data point (x_i, y_i) : $y_i - \hat{y}_i$

$$(\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i)$$

Residual sum of squares:
(RSS) $\sum_{i=1}^n (y_i - \hat{y}_i)^2$



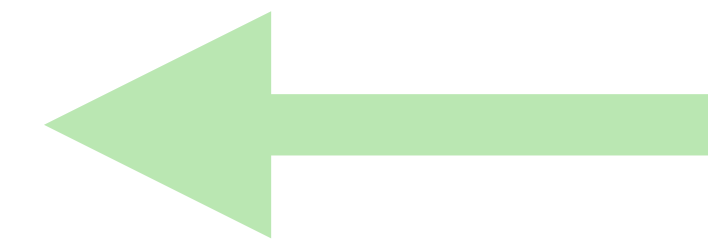
Find line that
minimizes RSS.

LINEAR REGRESSION

Residual of data point (x_i, y_i) : $y_i - \hat{y}_i$

$$(\hat{y}_i = \hat{w}_0 + \hat{w}_1 x_i)$$

Residual sum of squares:
(RSS) $\sum_{i=1}^n (y_i - \hat{y}_i)^2$



Find line that
minimizes RSS.

With one variable, X :

$$\hat{w}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

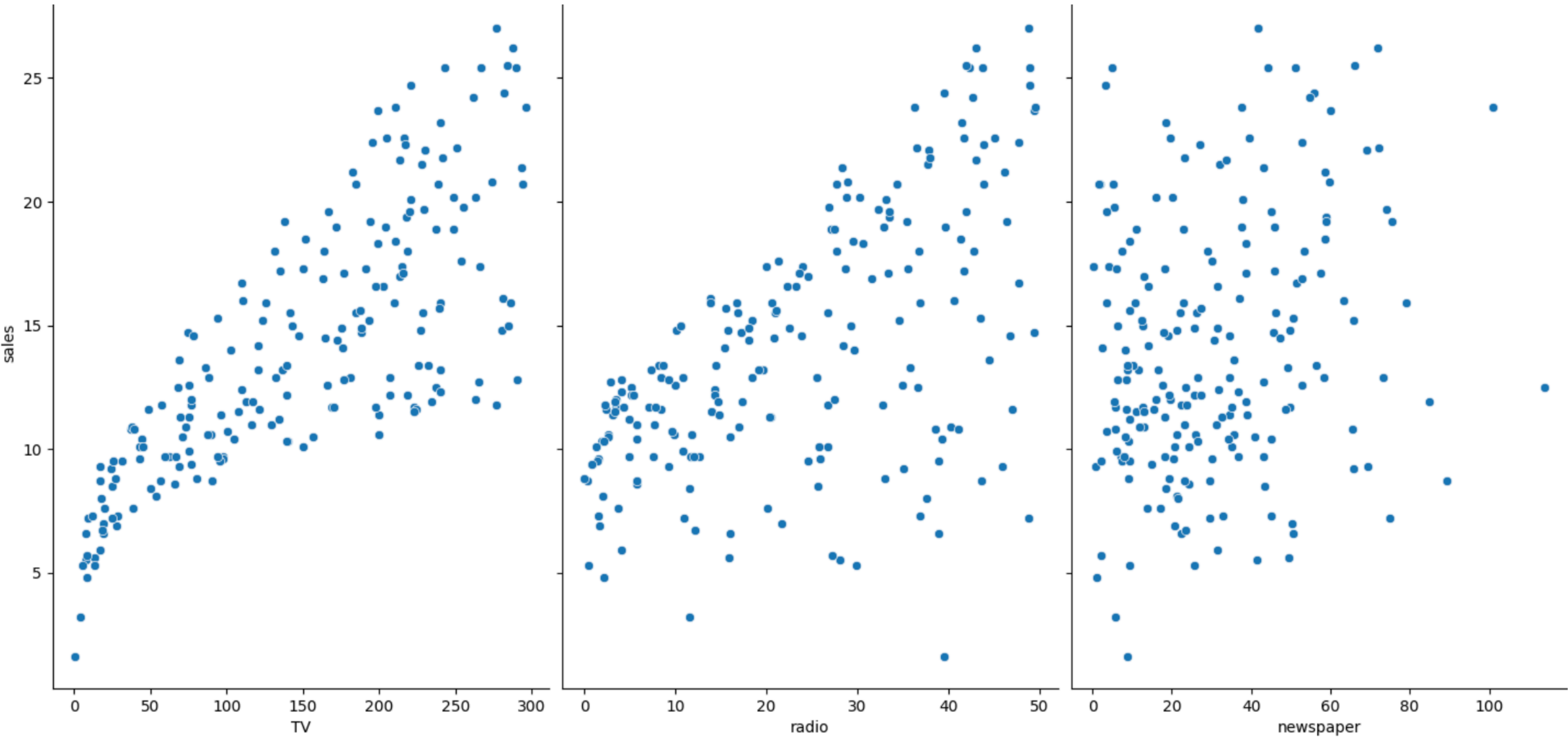
LINEAR REGRESSION

Ex: Advertising data set.

```
df.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

LINEAR REGRESSION



LINEAR REGRESSION

Linear regression only using TV feature.

	x	y
	TV	sales
0	230.1	22.1
1	44.5	10.4
2	17.2	9.3
3	151.5	18.5
4	180.8	12.9

Fit: Find $\hat{w}_0, \hat{w}_1$ such that $\sum_{i=1}^n (y_i - \hat{w}_0 - \hat{w}_1 x_i)^2$ is minimum.

LINEAR REGRESSION

```
X = df['TV'].values.reshape(-1, 1)
y = df.sales

X_train,X_test, y_train, y_test = \
    train_test_split(X,y, train_size=0.8, random_state=0)

# Fit one feature
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)

# Print coefficients
print(lr.intercept_)
print(lr.coef_)
```

```
7.292493773559364
[0.04600779]
```

LINEAR REGRESSION

```
X = df['TV'].values.reshape(-1, 1)
y = df.sales

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)

# Fit one feature
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)

# Print coefficients
print(lr.intercept_)
print(lr.coef_)
```

7.292493773559364 ← $\hat{w}_0$ Assuming no advertising, will sell approximately 7292 units.

[0.04600779] ← $\hat{w}_1$ Every additional \$1000 spent on TV advertising is associated with selling approximately 46 additional units.

LINEAR REGRESSION

More on coefficients: Accuracy of coefficient estimates

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

With statsmodels we need to add intercept explicitly.



LINEAR REGRESSION

More on coefficients: Accuracy of coefficient estimates

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

	coef	std err	t	P> t	[0.025	0.975]
const	7.2925	0.529	13.795	0.000	6.248	8.337
x1	0.0460	0.003	15.031	0.000	0.040	0.052

LINEAR REGRESSION

More on coefficients: Accuracy of coefficient estimates

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

	coef	std err	t	P> t	[0.025	0.975]
const	7.2925	0.529	13.795	0.000	6.248	8.337
x1	0.0460	0.003	15.031	0.000	0.040	0.052

Recall that:

$$\mu \in \bar{X}_n \pm k \times SE(\hat{\mu})$$

($k \approx 2$ for 95 % confidence interval)

LINEAR REGRESSION

More on coefficients: Accuracy of coefficient estimates

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

	coef	std err	t	P> t	[0.025	0.975]
const	7.2925	0.529	13.795	0.000	6.248	8.337
x1	0.0460	0.003	15.031	0.000	0.040	0.052

Recall that:

$$\mu \in \bar{X}_n \pm k \times SE(\hat{\mu})$$

($k \approx 2$ for 95 % confidence interval)

We can apply to $\hat{w}_0, \hat{w}_1$:

95 % confidence interval for w_0 is $\hat{w}_0 \pm 2 \times SE(\hat{w}_0)$

In our example (6.248,8.337)

LINEAR REGRESSION

Accuracy of the model: RSE measure

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

LINEAR REGRESSION

Accuracy of the model: *RSE* measure

```
import statsmodels.api as sm
X_train_sm = sm.add_constant(X_train)
lr_sm = sm.OLS(y_train, X_train_sm)
lr_sm_result = lr_sm.fit()
lr_sm_result.summary()
```

```
# Residual standard error
np.sqrt(lr_sm_result.scale)
```

```
3.2789684108159634
```

Residual standard error -
Average amount that the
response will deviate from
true regression line:

$$RSE = \sqrt{\frac{RSS}{n - 2}}$$

LINEAR REGRESSION

Accuracy of the model: R^2 statistic

```
X = df['TV'].values.reshape(-1, 1)
y = df.sales

X_train,X_test, y_train, y_test = \
    train_test_split(X,y, train_size=0.8, random_state=0)

# Fit one feature
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)
```

Back to lr model using
scikit learn

LINEAR REGRESSION

Accuracy of the model: R^2 statistic

```
X = df['TV'].values.reshape(-1, 1)
y = df.sales
```

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
```

```
# Fit one feature
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)
```

```
r2 = lr.score(X_train, y_train)
print('R-squared: ', r2)
```

```
R-squared: 0.5884742462828709
```

Fraction of
Variance
Unexplained

$$R^2 = 1 - \frac{\sum_i (y_i - f(x_i))^2}{\sum_i (y_i - \bar{y})^2}$$

Fraction of Variability of Y
explained using X .

LINEAR REGRESSION

Accuracy of the model: R^2 statistic

```
X = df['TV'].values.reshape(-1, 1)
y = df.sales

X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)

# Fit one feature
lr = linear_model.LinearRegression()
lr.fit(X_train, y_train)
```

```
r2 = lr.score(X_train, y_train)
print('R-squared: ', r2)
```

R-squared: 0.5884742462828709

```
np.corrcoef(X_train[:,0], y_train.to_numpy())
```

```
array([[1., 0.76712075],
       [0.76712075, 1.]])
```

R^2 same as square of correlation coefficient in simple linear regression.

MULTIPLE LINEAR REGRESSION

Not just one feature x , but instead m features: $x_1, x_2, \dots, x_m$

Ex: Predicting **sales** from 2 features and 3 features.

MULTIPLE LINEAR REGRESSION

Not just one feature x , but instead m features: $x_1, x_2, \dots, x_m$

Ex: Predicting **sales** from 2 features and 3 features.

```
df.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

Let's start with 3 features.

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3$$

Predicted sales Intercept TV radio newspaper

MULTIPLE LINEAR REGRESSION

Not just one feature x , but instead m features: $x_1, x_2, \dots, x_m$

Ex: Predicting sales from 2 features and 3 features.

```
df.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

Let's start with 3 features.

$$\hat{y} = \hat{w}_0 + \hat{w}_1 x_1 + \hat{w}_2 x_2 + \hat{w}_3 x_3$$

Diagram illustrating the components of the multiple linear regression equation:

- $\hat{y}$: Predicted sales (indicated by a green arrow)
- $\hat{w}_0$: Intercept (indicated by a red arrow)
- $\hat{w}_1 x_1$: TV (indicated by a green arrow)
- $\hat{w}_2 x_2$: radio (indicated by a green arrow)
- $\hat{w}_3 x_3$: newspaper (indicated by a green arrow)

Ex: Say row 4 is not in training data and we want to predict it. $\hat{y}_4 = \hat{w}_0 + \hat{w}_1 180.8 + \hat{w}_2 10.8 + \hat{w}_3 58.4$ where $y_4 = 12.9$

MULTIPLE LINEAR REGRESSION

Apply 5-fold cross validation and predict from all 3 features.

```
# 5-fold cross validation
X = df[['TV', 'radio', 'newspaper']]
y = df.sales
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
cvs = cross_val_score(linear_model.LinearRegression(),
                      X_train, y_train, cv = 5, scoring='r2')
print('Mean R2: ', cvs.mean())
```

Mean R2: 0.8985145014718592

MULTIPLE LINEAR REGRESSION

Apply 5-fold cross validation and predict from all 3 features.

```
# 5-fold cross validation
X = df[['TV', 'radio', 'newspaper']]
y = df.sales
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
cvs = cross_val_score(linear_model.LinearRegression(),
                      X_train, y_train, cv = 5, scoring='r2')
print('Mean R2: ', cvs.mean())
```

Mean R2: 0.8985145014718592

Better R^2 than only using TV feature.

How about 2 features?

MULTIPLE LINEAR REGRESSION

Apply 5-fold cross validation and predict from TV and radio.

```
X = df[['TV', 'radio']]
y = df.sales
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
cvs = cross_val_score(linear_model.LinearRegression(),
                      X_train, y_train, cv = 5, scoring='r2')
print('Mean R2: ', cvs.mean())
```

Mean R2: 0.9010773153077544

MULTIPLE LINEAR REGRESSION

Apply 5-fold cross validation and predict from TV and radio.

```
X = df[['TV', 'radio']]
y = df.sales
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
cvs = cross_val_score(linear_model.LinearRegression(),
                      X_train, y_train, cv = 5, scoring='r2')
print('Mean R2: ', cvs.mean())
```

Mean R2: 0.9010773153077544

Better R^2 than using all 3 features.

Fit using these 2 features and evaluate on the test set with mean RSS .

MULTIPLE LINEAR REGRESSION

Fit using TV and radio. Evaluate on the test set with mean RSS .

```
X = df[['TV', 'radio']]
y = df.sales
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, train_size=0.8, random_state=0)
lr_tr = linear_model.LinearRegression()
lr_tr.fit(X_train, y_train)
lr_tr_predict = lr_tr.predict(X_test)
# mse
lr_tr_mse = mean_squared_error(y_test, lr_tr_predict)
print('MSE : ', lr_tr_mse)
```

MSE : 4.391429763581883

MULTIPLE LINEAR REGRESSION

Dataset

x_{11}	x_{12}	$\cdots$	x_{1m}	y_1
x_{21}	x_{22}		x_{2m}	y_2
		$\vdots$		
x_{n1}	x_{n2}		x_{nm}	y_n

How is multiple linear regression formulated and solved in general?

Ex

```
df.head()
```

	TV	radio	newspaper	sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	9.3
3	151.5	41.3	58.5	18.5
4	180.8	10.8	58.4	12.9

MULTIPLE LINEAR REGRESSION

Dataset

x_{11}	x_{12}	$\cdots$	x_{1m}	y_1
x_{21}	x_{22}		x_{2m}	y_2
		$\vdots$		
x_{n1}	x_{n2}		x_{nm}	y_n

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_mx_m$$

$$\hat{y}_1 = w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_mx_{1m}$$

$$\hat{y}_2 = w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_mx_{2m}$$

$\vdots$

$$\hat{y}_n = w_0 + w_1x_{n1} + w_2x_{n2} + \cdots + w_mx_{nm}$$

MULTIPLE LINEAR REGRESSION

Dataset

x_{11}	x_{12}	$\cdots$	x_{1m}	y_1
x_{21}	x_{22}		x_{2m}	y_2
		$\vdots$		
x_{n1}	x_{n2}		x_{nm}	y_n

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_mx_m$$

$$\hat{y}_1 = w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_mx_{1m}$$

$$\hat{y}_2 = w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_mx_{2m}$$

$\vdots$

$$\hat{y}_n = w_0 + w_1x_{n1} + w_2x_{n2} + \cdots + w_mx_{nm}$$

How to turn it into nice matrix form?

MULTIPLE LINEAR REGRESSION

$\hat{y}$

$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_n$

=

X

1	x_{11}	x_{12}	$\cdots$	x_{1m}
1	x_{21}	x_{22}		x_{2m}
1			$\vdots$	
1	x_{n1}	x_{n2}		x_{nm}

w

w_0
w_1
w_m

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_mx_m$$

$$\hat{y}_1 = w_0 + w_1x_{11} + w_2x_{12} + \cdots + w_mx_{1m}$$

$$\hat{y}_2 = w_0 + w_1x_{21} + w_2x_{22} + \cdots + w_mx_{2m}$$

$$\vdots$$

$$\hat{y}_n = w_0 + w_1x_{n1} + w_2x_{n2} + \cdots + w_mx_{nm}$$

How to turn it into nice matrix form?

MULTIPLE LINEAR REGRESSION

$\hat{y}$

$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_n$

=

		X				
	1	x_{11}	x_{12}	$\cdots$	x_{1m}	w_0
	1	x_{21}	x_{22}		x_{2m}	w_1
	1			$\vdots$		
	1	x_{n1}	x_{n2}		x_{nm}	w_m

w

Note these are
the actual values

y

y_1
y_2
y_n

MULTIPLE LINEAR REGRESSION

$\hat{y}$

$\hat{y}_1$
$\hat{y}_2$
$\hat{y}_n$

=

X

1	x_{11}	x_{12}	$\cdots$	x_{1m}
1	x_{21}	x_{22}		x_{2m}
1			$\vdots$	
1	x_{n1}	x_{n2}		x_{nm}

w

w_0
w_1
w_m

Note these are
the actual values

y

y_1
y_2
y_n

We know X, y . How to find w minimizing:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

MULTIPLE LINEAR REGRESSION

Method 1: Algebraic closed form formula.

Theorem: Vector w minimizing mean squared error is

$$(X^T X)^{-1} X^T y$$

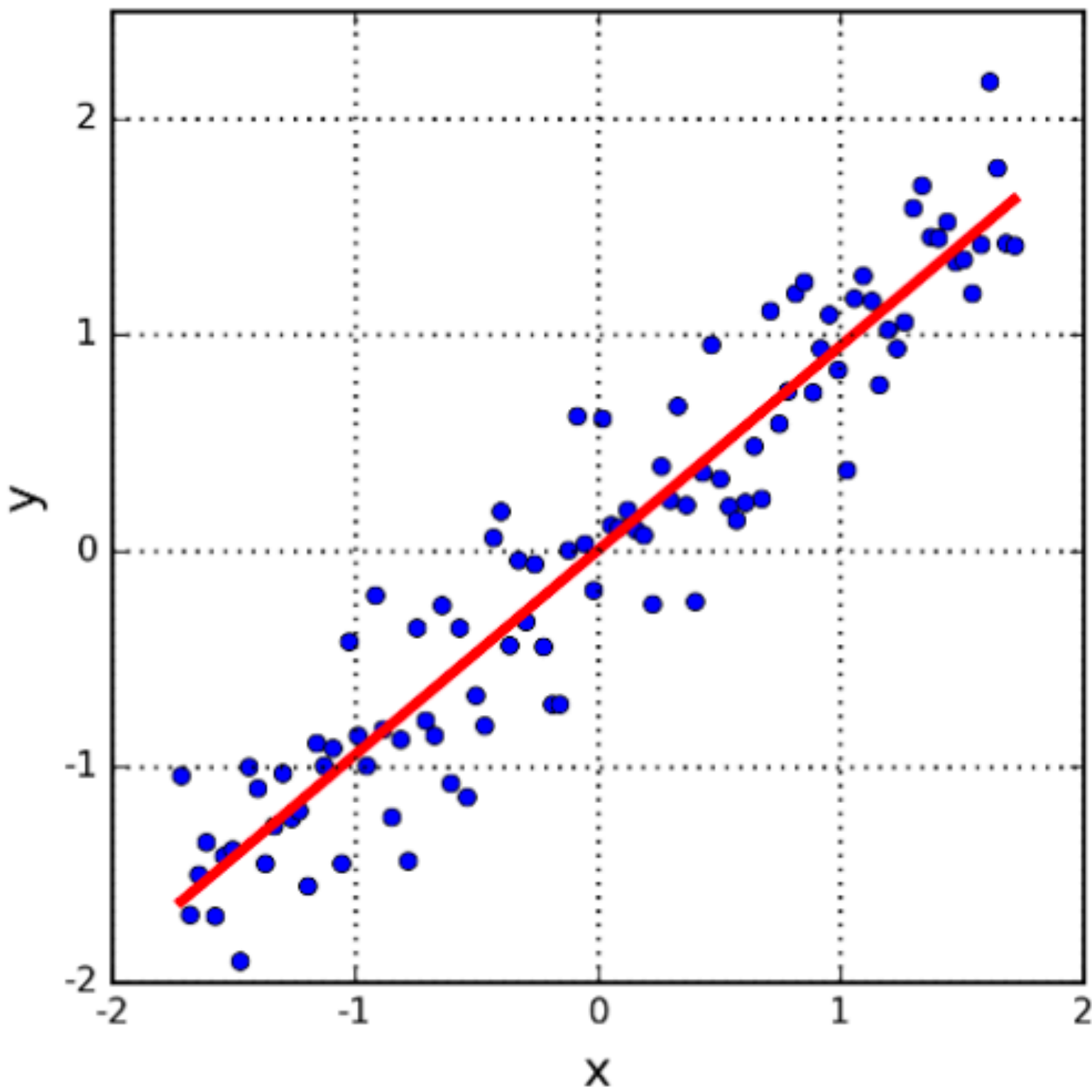
Might be problematic in practice: Matrix inversion slow, prone to numerical stability. Hard to generalize to other optimization problems.

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

Simple setting: Just one variable and intercept is 0: $\hat{y} = w_1 x$

What does the squared error function $\sum_{i=1}^n (y_i - \hat{y}_i)^2$ look like?

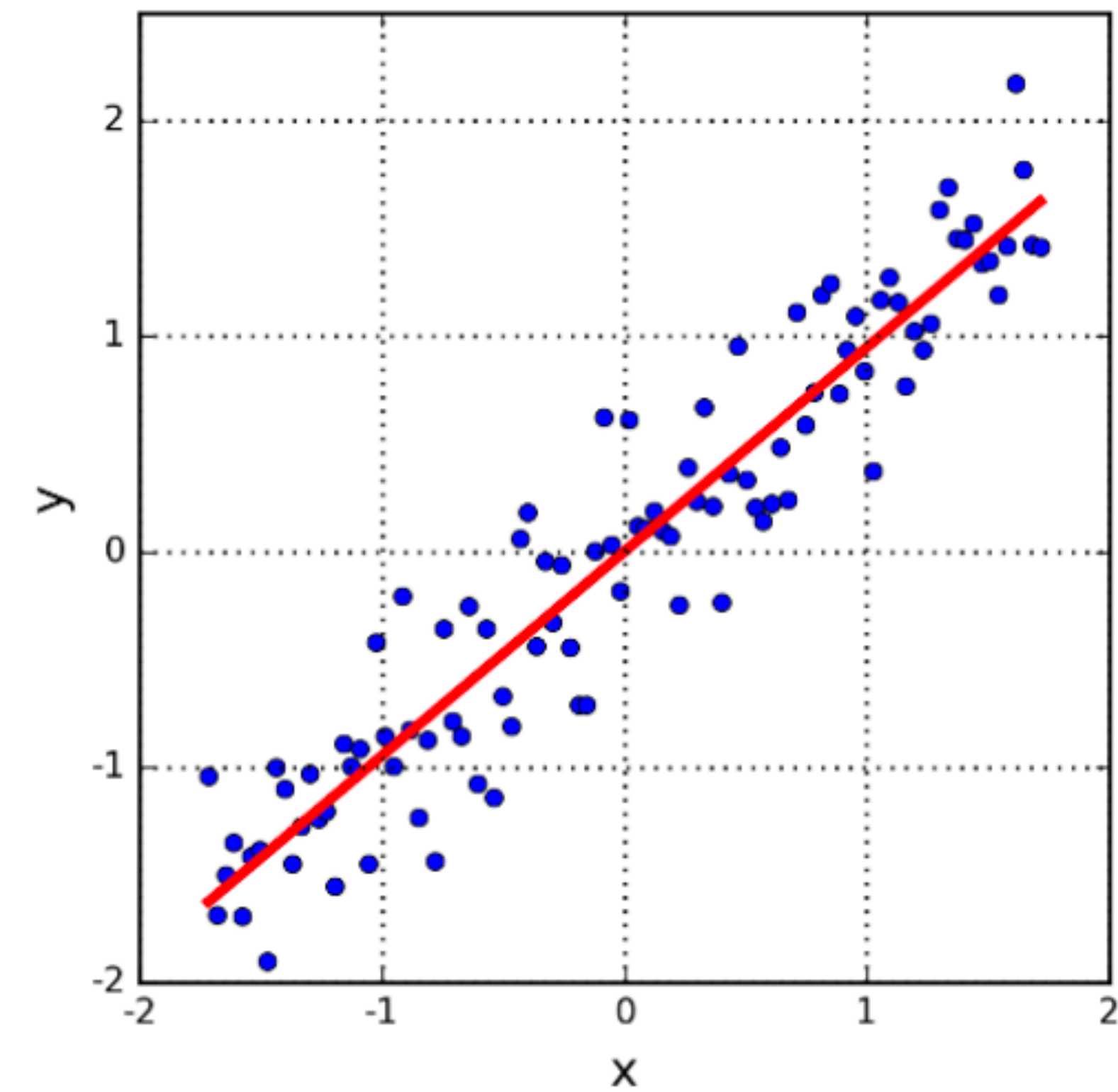


MULTIPLE LINEAR REGRESSION

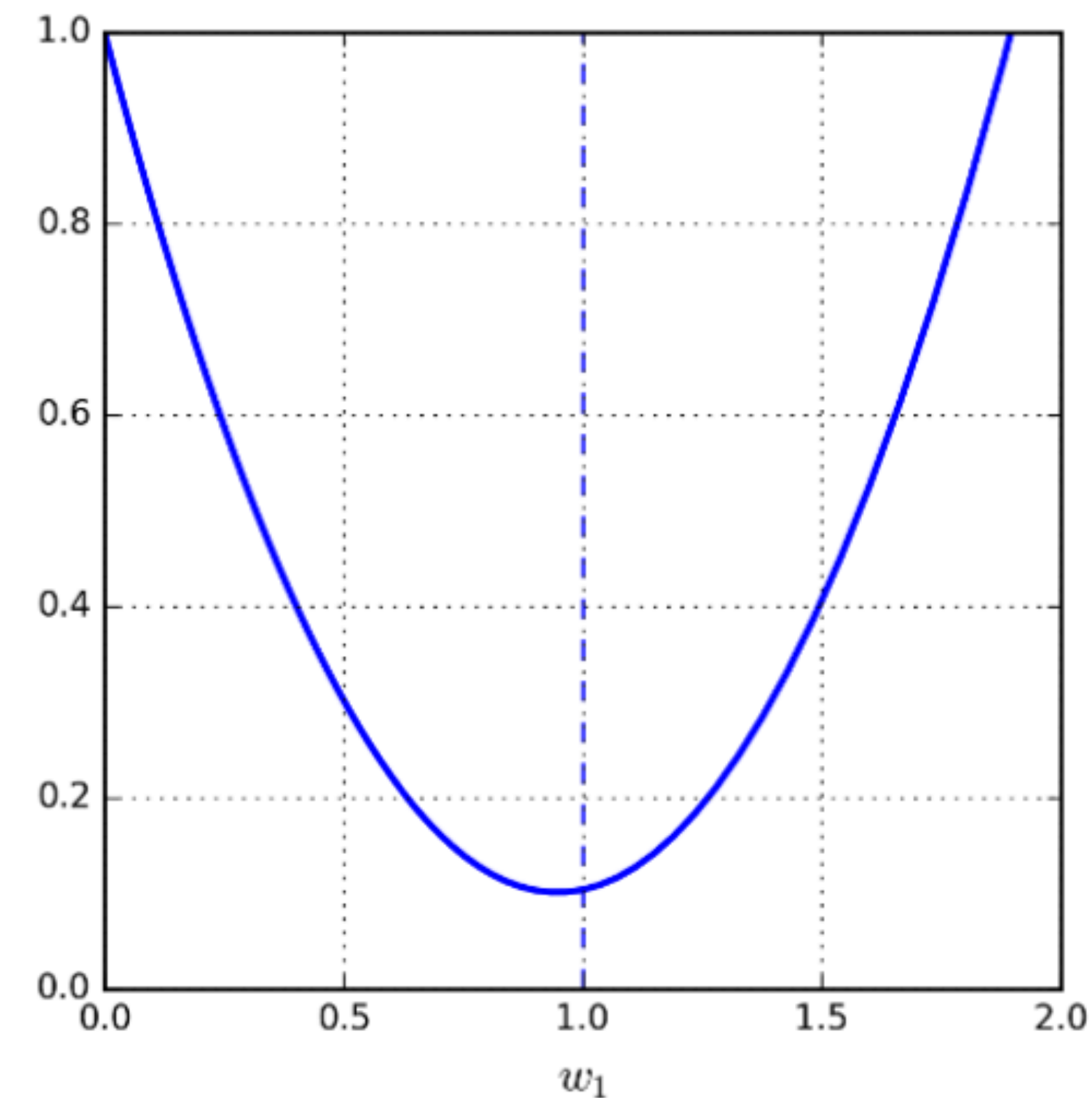
Method 2: Parameter fitting via Gradient Descent.

Simple setting: Just one variable and intercept is 0: $\hat{y} = w_1 x$

What does the squared error function $\sum_{i=1}^n (y_i - \hat{y}_i)^2$ look like?

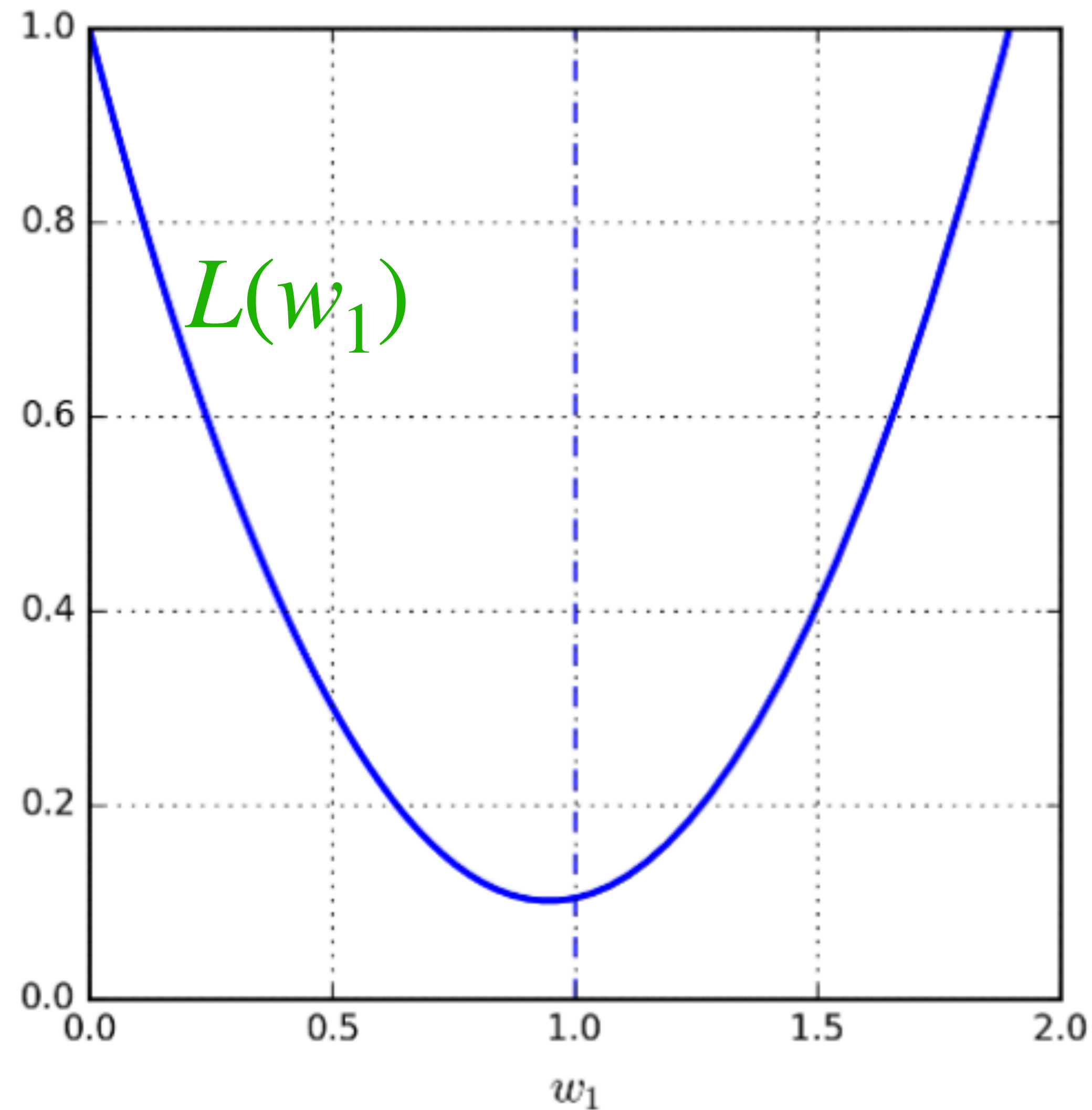


Quadratic function
of w_1 . Call it $L(w_1)$.
Easy to find the
minimum point.



MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

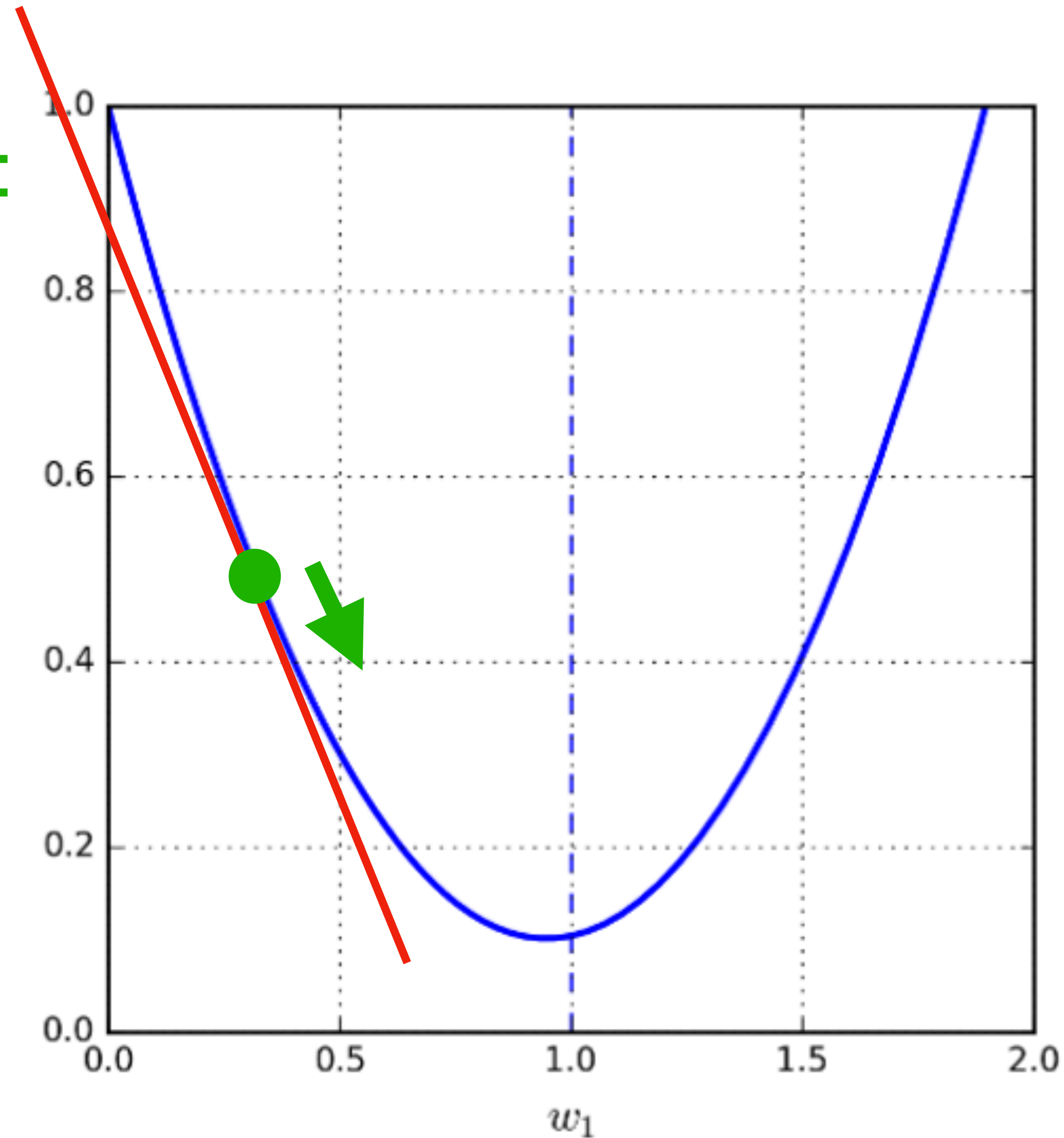


Start somewhere
move wrt to the
slope (derivative)

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

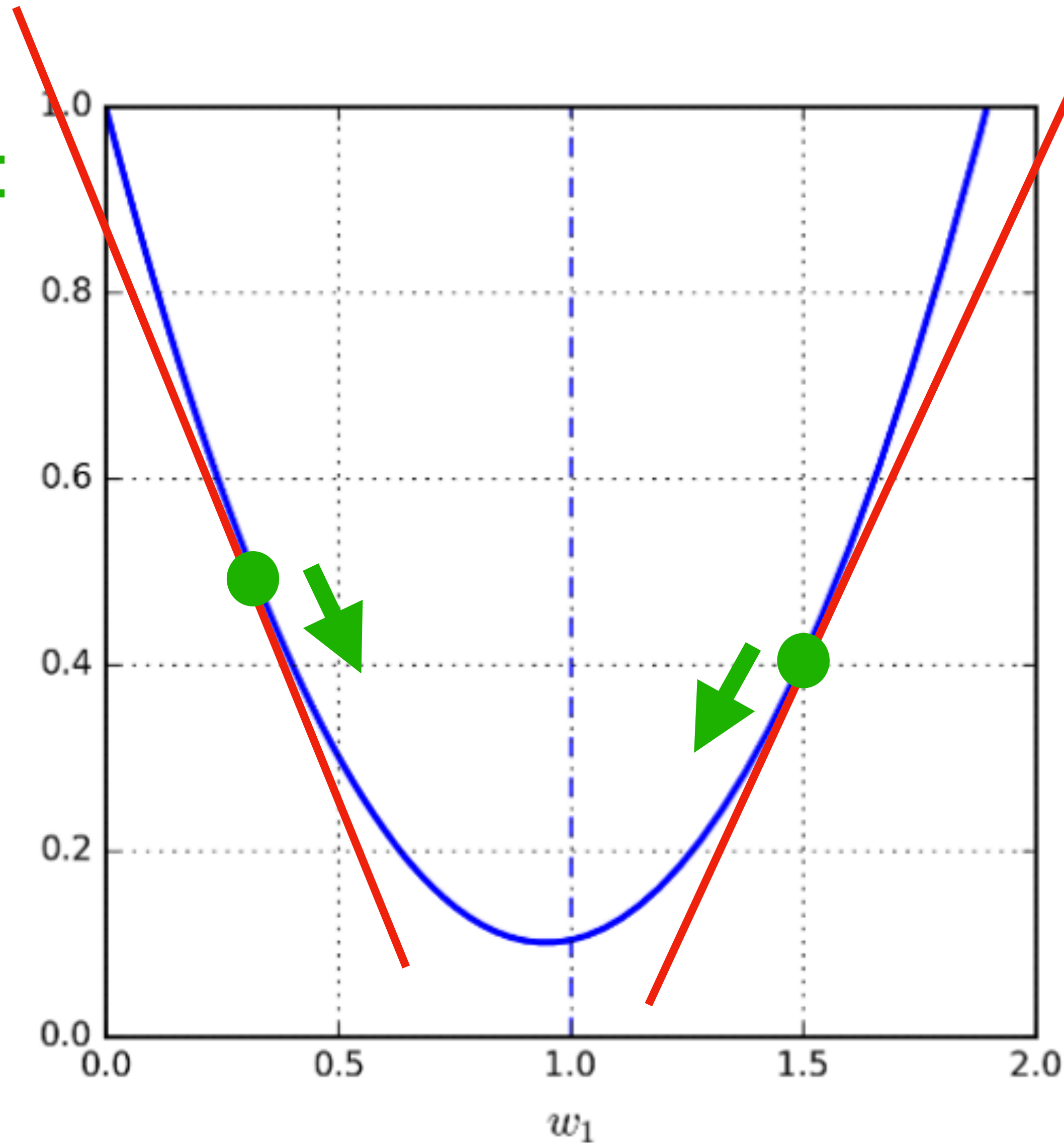
Negative slope:
Move in the
positive
direction.



MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

Negative slope:
Move in the
positive
direction.



Positive slope:
Move in the
negative
direction.

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

Repeat until convergence:

$$w_1^{t+1} = w_1^t - \frac{d}{dw_1} L(w_1)$$

Not power! Value of w_1 after t iterations.

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

Repeat until convergence:

$$w_1^{t+1} = w_1^t - \frac{d}{dw_1} L(w_1)$$

Next guess for w

Current w

Move opposite to the slope

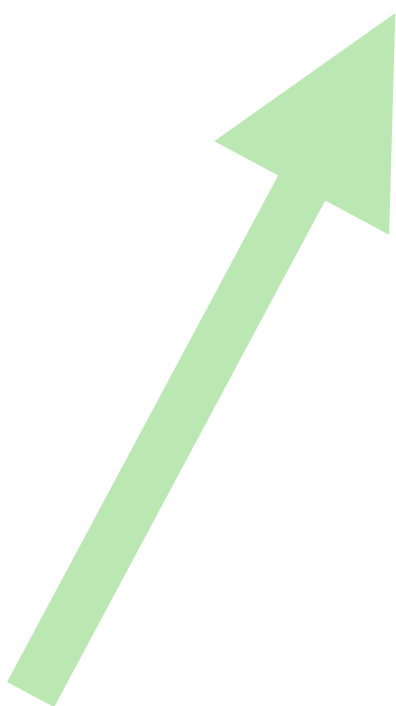
MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

Repeat until convergence:

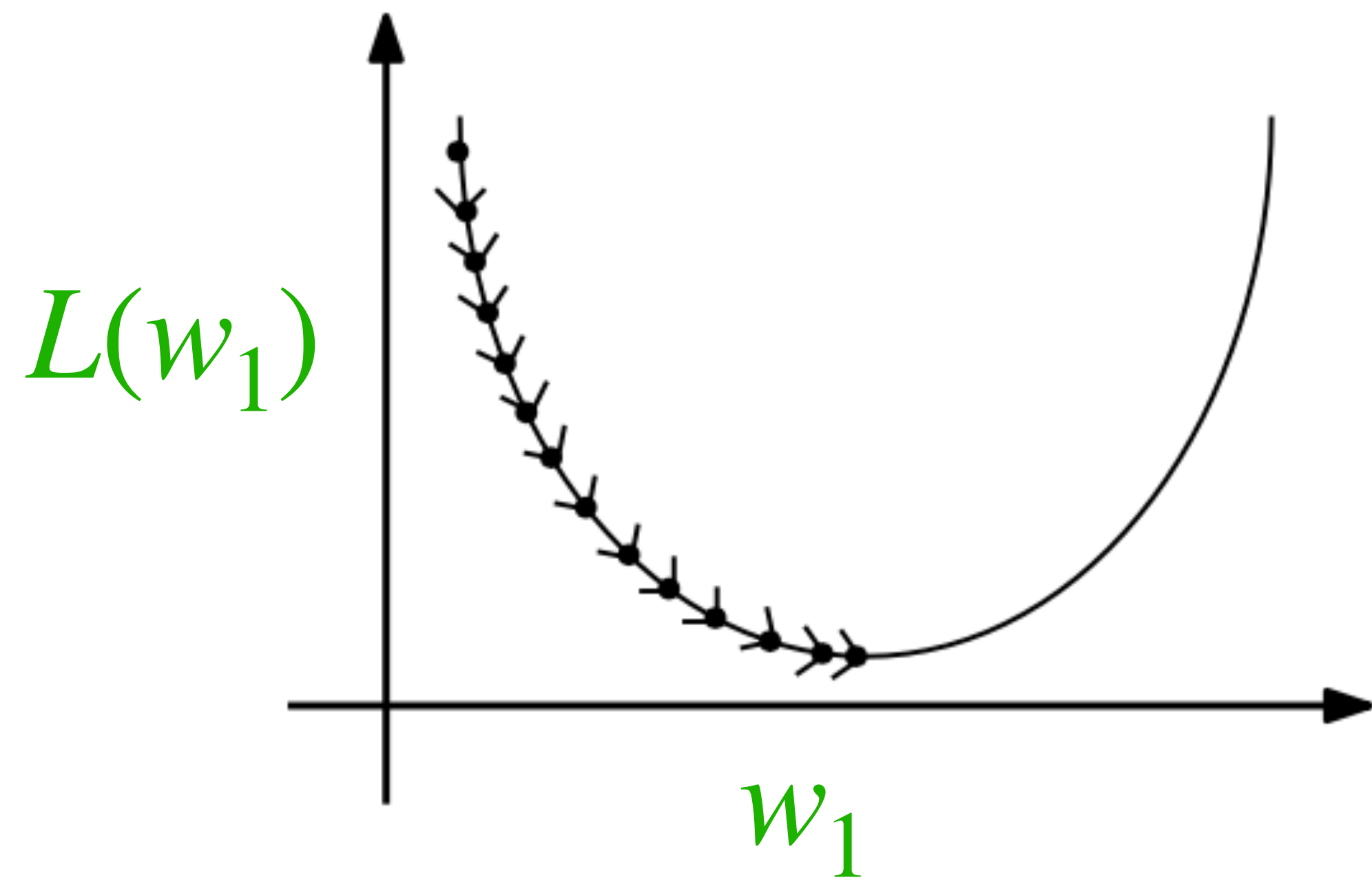
$$w_1^{t+1} = w_1^t - \alpha \frac{d}{dw_1} L(w_1)$$

How large steps? Introduce α , learning rate, for step size.

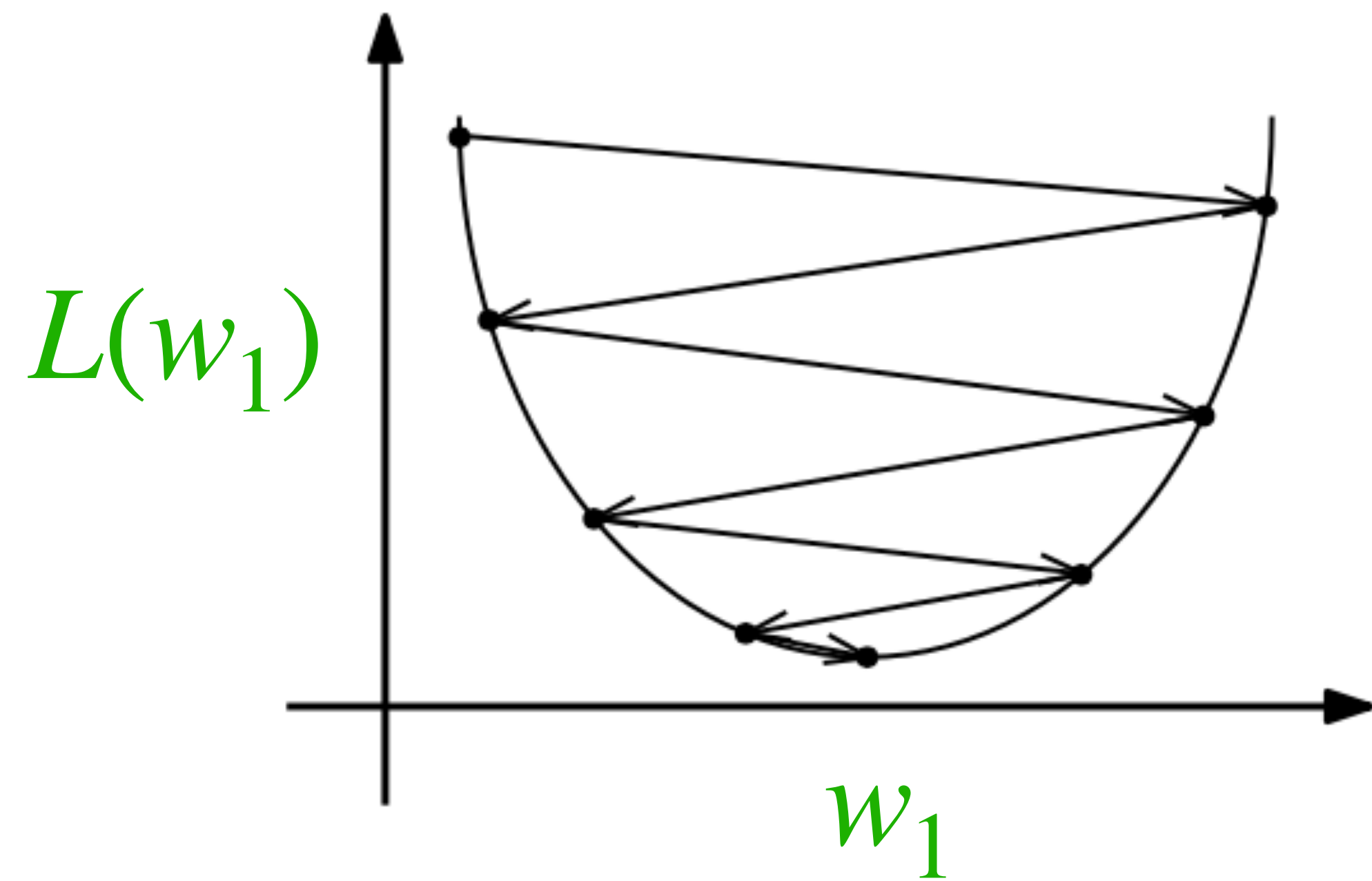


MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.



Small α may converge slowly.



Large α may overshoot.

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

What happens with multiple dimensions (more than one)?

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

What happens with multiple dimensions (more than one)?

Repeat until convergence:

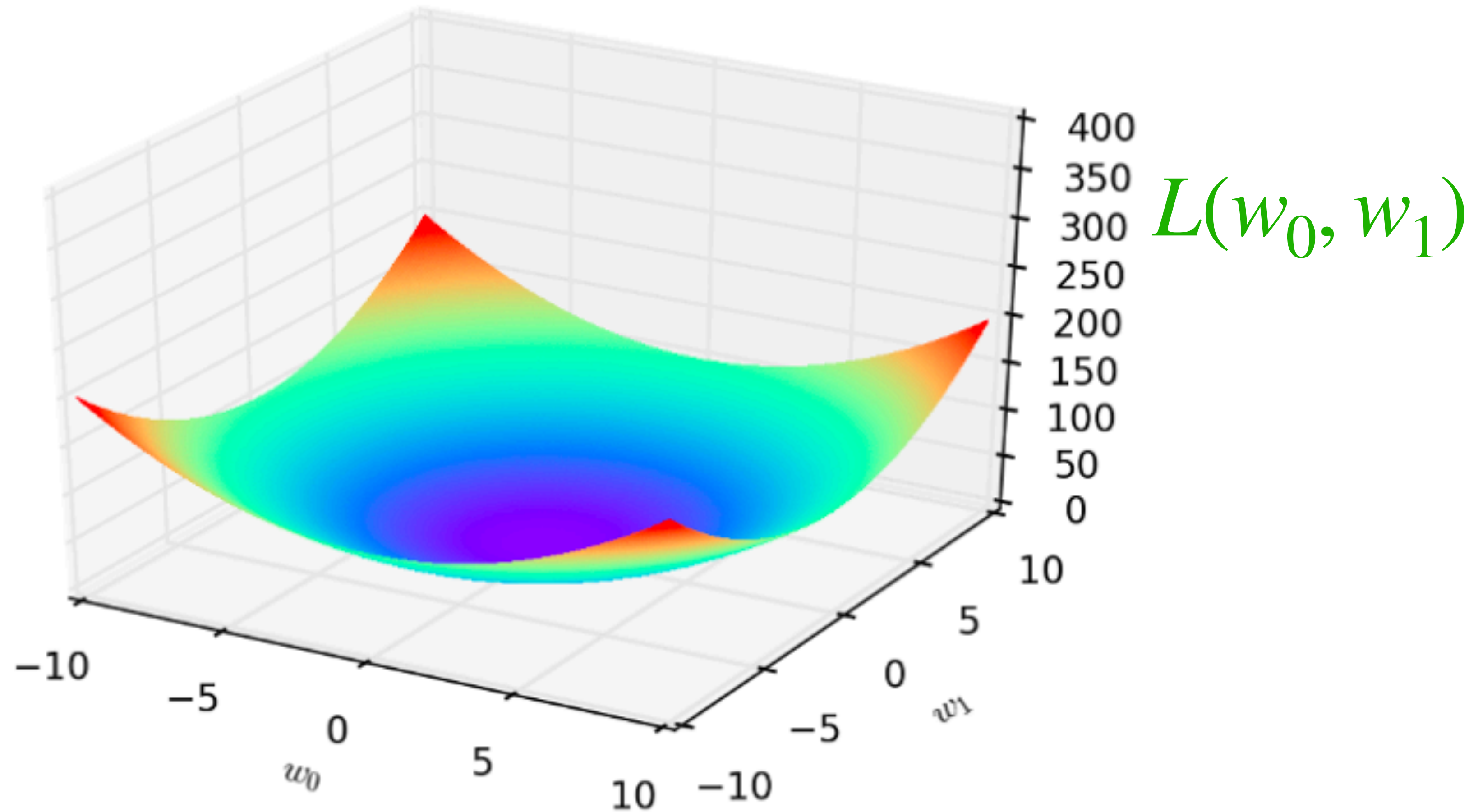
$$\begin{bmatrix} w_0^{t+1} \\ w_1^{t+1} \\ \vdots \\ w_m^{t+1} \end{bmatrix} = \begin{bmatrix} w_0^t \\ w_1^t \\ \vdots \\ w_m^t \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_m} \end{bmatrix}$$

Why does it still work?

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.

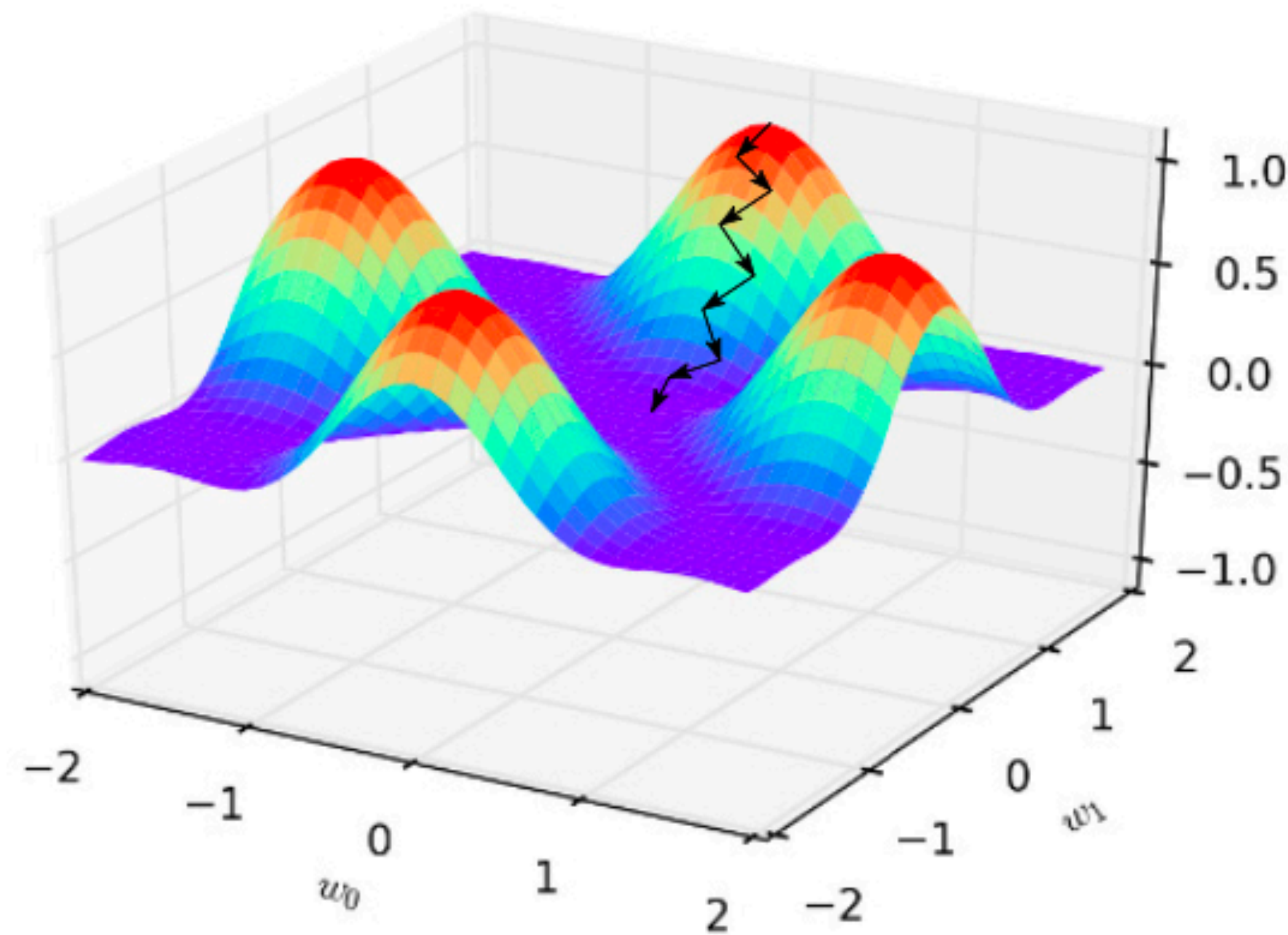
What happens with multiple dimensions (more than one)?



Why does it still work? Loss function remains **convex** no matter what m .

MULTIPLE LINEAR REGRESSION

Method 2: Parameter fitting via Gradient Descent.



$L(w_0, w_1)$

If the function to optimize is not convex? Gradient descent may not find the global optimum, but will find some local minima.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

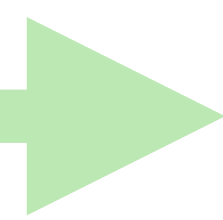
Problem with Method 2, Gradient Descent:

$$\begin{bmatrix} w_0^{t+1} \\ w_1^{t+1} \\ \vdots \\ w_m^{t+1} \end{bmatrix} = \begin{bmatrix} w_0^t \\ w_1^t \\ \vdots \\ w_m^t \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_m} \end{bmatrix}$$

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Problem with Method 2, Gradient Descent:

$$\begin{bmatrix} w_0^{t+1} \\ w_1^{t+1} \\ \vdots \\ w_m^{t+1} \end{bmatrix} = \begin{bmatrix} w_0^t \\ w_1^t \\ \vdots \\ w_m^t \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_m} \end{bmatrix}$$



Each partial derivative hides a lot of computations in the form of a sum:

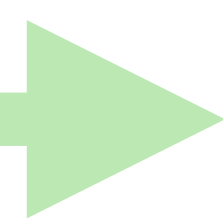
$$\frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j} \sum_{i=1}^n (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_m x_{im}))^2$$

For each dimension $j \Rightarrow$ go through all n training points.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Problem with Method 2, Gradient Descent:

$$\begin{bmatrix} w_0^{t+1} \\ w_1^{t+1} \\ \vdots \\ w_m^{t+1} \end{bmatrix} = \begin{bmatrix} w_0^t \\ w_1^t \\ \vdots \\ w_m^t \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_0} \\ \frac{\partial L}{\partial w_1} \\ \vdots \\ \frac{\partial L}{\partial w_m} \end{bmatrix}$$



Each partial derivative hides a lot of computations in the form of a sum:

$$\frac{\partial L}{\partial w_j} = \frac{\partial}{\partial w_j} \sum_{i=1}^n (y_i - (w_0 + w_1 x_{i1} + w_2 x_{i2} + \cdots + w_m x_{im}))^2$$

For each dimension $j \Rightarrow$ go through all n training points.

Computationally expensive if too many training points.

Solution: Mini-batch or stochastic gradient descent.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Mini-batch gradient descent

Use a subset of the data at each update.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Mini-batch gradient descent

Use a subset of the data at each update.

One training epoch:

Compute gradient on the first $x\%$ of the data. Update w_j , for $\forall w_j$

Compute gradient on the next $x\%$ of the data. Update w_j , for $\forall w_j$

...

Compute gradient on the last $x\%$ of the data. Update w_j , for $\forall w_j$

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Mini-batch gradient descent

Use a subset of the data at each update.

One training epoch:

Compute gradient on the first $x\%$ of the data. Update w_j , for $\forall w_j$

Compute gradient on the next $x\%$ of the data. Update w_j , for $\forall w_j$

...

Compute gradient on the last $x\%$ of the data. Update w_j , for $\forall w_j$

In a single training epoch, we use every datapoint in the data once.

Several training epochs are performed as necessary.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Stochastic gradient descent

Extreme case of mini-batch: Each subset is a single datapoint.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT

Stochastic gradient descent

Extreme case of mini-batch: Each subset is a single datapoint.

One training epoch:

Compute gradient on the **first datapoint**. Update w_j , for $\forall w_j$

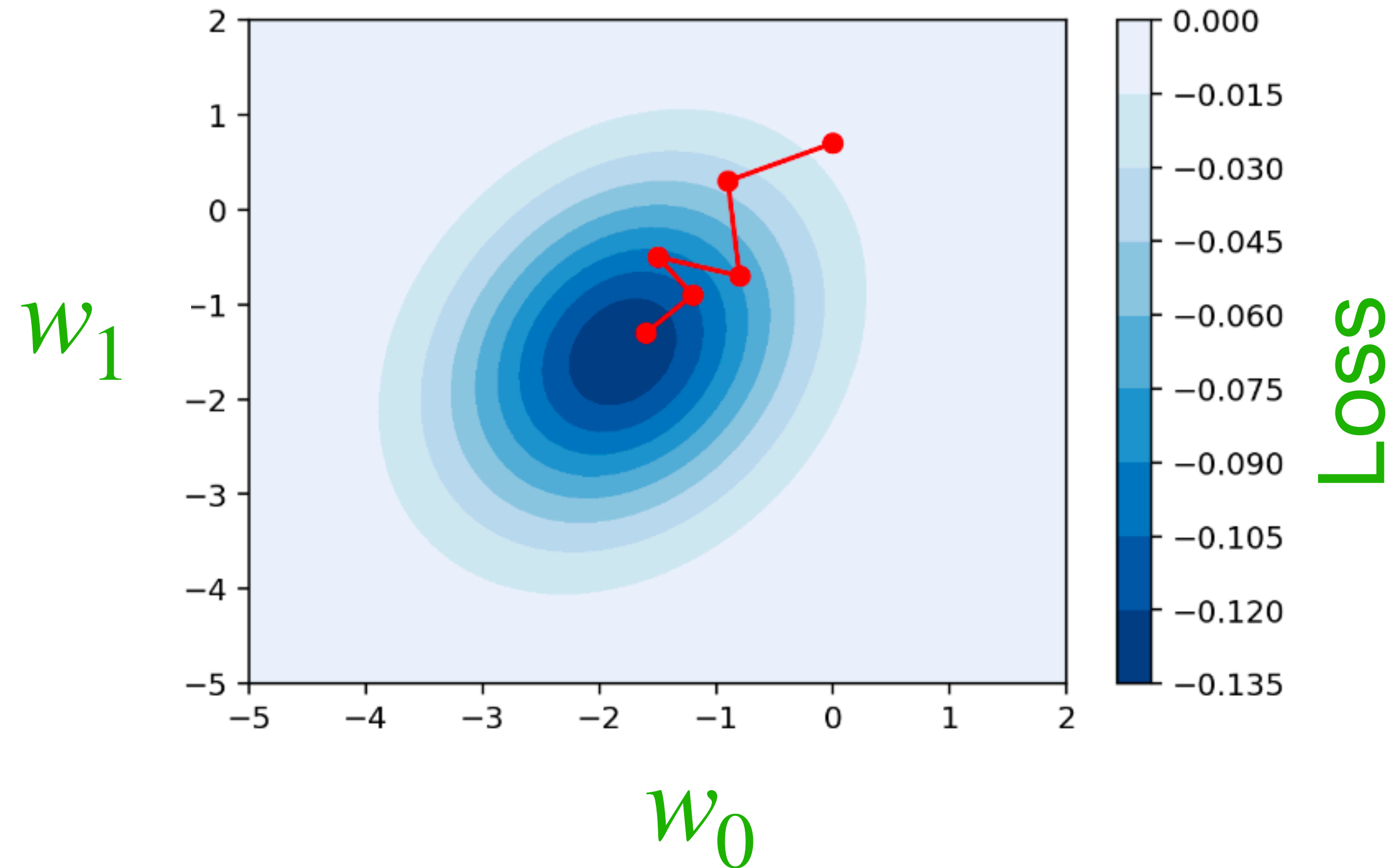
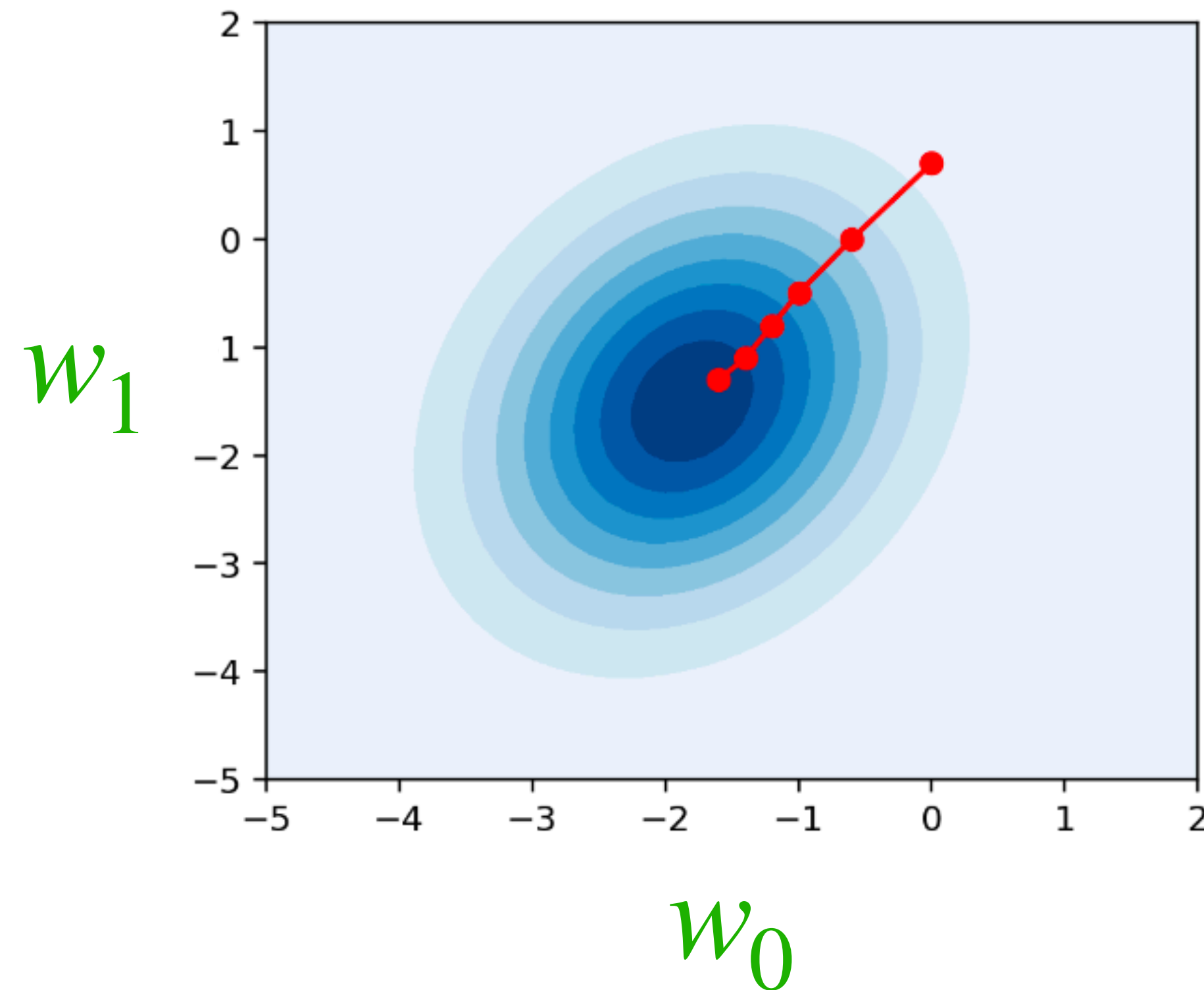
Compute gradient on the **next datapoint**. Update w_j , for $\forall w_j$

...

Compute gradient on the **last datapoint**. Update w_j , for $\forall w_j$

Several training epochs are performed as necessary.

MINI BATCH AND STOCHASTIC GRADIENT DESCENT



Gradient descent:

- True gradient
- Always descends toward true minimum loss.

Mini-batch/stochastic gradient descent:

- Approximation of true gradient
- May not descend toward true minimum loss with each update.